

Table of Contents

1. Executive Summary & Overview	Page 2
2. Tech Stack & Workspace Structure	Page 3
3. Database Schema & Migration Management	Page 4
4. Backend Services Layer Deep-Dive	Page 5
5. Frontend Layout & Interactive Components	Page 6
6. Local Execution & Environment Setup	Page 7

1. Executive Summary & Overview

Tech Decision is a specialized web application designed to help consumers navigate the complex smartphone market. By tracking prices, crawling detailed specs, normalising raw configuration matrices, and validating commercial discounts, it provides users with honest, data-driven purchasing verdicts. The system aims to protect consumers from 'fake discounts'—a widespread practice where online retailers temporarily inflate listed prices (MRPs) just before sales to make discount percentages appear far higher than they actually are.

Key capabilities of the system include:

- **Specs Normalization:** Automatically translates messy, inconsistent raw specifications fetched from mobile sites into clean data types (e.g. converting '5000 mAh' string to an integer).
- **Variant Extraction:** Recognises distinct product combinations based on RAM, storage capacity, and color, and maps pricing details directly to those variants.
- **Seller Trust Scoring:** Evaluates listing platforms (Amazon, Flipkart, Croma, Reliance) and independent merchants to guide buyers to reputable sources.
- **AI Verdict Generation:** Connects with LLMs to generate readable breakdowns, pros, cons, and buying recommendations.

2. Tech Stack & Workspace Structure

The project is structured as a full-stack, split-folder repository. The frontend is deployed to Vercel and handles browser routing and responsive layouts, while the backend API is a FastAPI service that performs the business logic, crawlers, calculations, and manages DB state.

Technology Stack Matrix

Layer	Technology / Libraries Used	Key Responsibilities
Frontend	Next.js 16 (App Router), TypeScript, Tailwind CSS	Search Interface, variant selector, responsive charts, dynamic s
Backend	FastAPI, Uvicorn, SQLAlchemy (ORM), Alembic, Pydantic	REST API, Data Parser, Request normalization engine, buying d
Database	PostgreSQL (Production) / SQLite (Local)	Persistent tables for phones, specs, listings, AI insights, and va
AI Layer	OpenAI API (GPT-4 / GPT-3.5)	Summarizes battery, display, camera, and computes honest pu

Repository Workspace Layout

```
c:/Users/SAMSUNG/OneDrive/Desktop/My-Projects/33/
■■■ README.md                <- Root readme file
■■■ .gitignore                <- Root git ignore settings
■■■ tech-decision/           <- Project root folder
    ■■■ backend/             <- FastAPI Application
        ■■■ main.py           <- API entry point & CORS middleware
        ■■■ requirements.txt   <- Python dependencies list
        ■■■ alembic.ini        <- Alembic database migrations configuration
        ■■■ alembic/           <- Migration script files
        ■■■ app/
            ■■■ api/           <- Routers (health.py, phones.py, discovery.py)
            ■■■ core/           <- Config settings (.env loaders)
            ■■■ db/             <- Session creators & Declarative Base
            ■■■ models/         <- SQLAlchemy db schemas
            ■■■ parsers/        <- GSMArena spec scraper & merchant scrapers
            ■■■ schemas/        <- Pydantic response models
            ■■■ services/       <- Business logic & Engines (pricing, decision)
    ■■■ frontend/            <- Next.js 16 Web Application
        ■■■ package.json       <- Node dependencies list
        ■■■ next.config.mjs     <- Next.js specific compiler settings
        ■■■ tailwind.config.ts  <- Custom colors, layout & font tokens
        ■■■ components/        <- Shared UI parts (Search hero, UI inputs)
        ■■■ app/
            ■■■ layout.tsx      <- Global wrapper & viewport configurations
            ■■■ page.tsx        <- Entry point / Search view
            ■■■ phones/[slug]/  <- Dynamic detailed product page
```

3. Database Schema & Migration Management

Persistence is designed around an ORM mapped layer using SQLAlchemy. The system supports full relational foreign key mappings with cascading deletes. Migrations are managed with Alembic, enabling smooth schema upgrades (e.g. moving from single-price listings to variant-specific pricing matrices).

Entity-Relationship Overview

- **phones:** Core directory table storing model name, brand, slug, and launch pricing.
- **phone_specs:** Holds normalized specs (ram_gb, storage_gb, chipset, charging_watts, display_size, refresh_rate_hz) and the raw crawled string fields.
- **phone_insights:** Stores the AI summaries generated via OpenAI.
- **phone_variants:** Contains mapped SKUs of RAM/Storage/Color combinations.
- **variant_prices:** Mapped store-specific listings (Amazon, Flipkart) tracking MRPS and final prices for distinct variants.
- **price_listings:** Mapped listings for base-model legacy mappings.

Database Table Columns Reference

Table Name	Column	Data Type	Constraints / Key
phones	id	UUID	PRIMARY KEY, Default UUIDv4
	slug	String(210)	UNIQUE, INDEX
	brand, model	String	INDEX
phone_specs	id	UUID	PRIMARY KEY
	phone_id	UUID	FOREIGN KEY (phones.id), UNIQUE
	battery_mah	Integer	Nullable
	refresh_rate_hz	Integer	Nullable
phone_variants	display_size	Float	Nullable
	id	UUID	PRIMARY KEY
	phone_id	UUID	FOREIGN KEY (phones.id)
	ram_gb, storage_gb	Integer	NotNull
variant_prices	id	UUID	PRIMARY KEY
	variant_id	UUID	FOREIGN KEY (phone_variants.id)
	platform	String(100)	NotNull (Amazon, Flipkart)
	listed_price, final_price	Integer	NotNull
	fake_discount_flag	Boolean	Default False

4. Backend Services Layer Deep-Dive

The core intelligence of Tech Decision resides in its modular backend service engines. These services are isolated in the ``app/services`` directory, decoupling data models from business logic.

Pricing Engine & Fake Discount Detection

The **PricingService** aggregates prices dynamically if listings are older than 1 hour. It leverages web parsers (``AmazonParser``, ``FlipkartParser``, etc.) to extract raw listed price and MRP. The **FakeDiscountDetector** then analyzes the difference: if the platform-stated MRP is significantly higher than the launch price or previous averages, it flags a possible 'fake discount' and recalculates the *discount authenticity score*. It deducts coupon and bank cashbacks to compute the true **Best Effective Price**.

Decision Engine Verdict Classifier

The **DecisionEngine** calculates a value score out of 100 based on hardware specs, adjusted by RAM/storage upgrades on variants. It then matches these against competitors' indexes and historic price fluctuations. The decision is categorized into one of four states:

Verdict State	Criteria	Typical Recommendation Summary
BUY_NOW	Price is near historical low and specs are legitimate.	Excellent time to buy — price is at its absolute lowest.
WAIT_FOR_PRICE_DROP	Current platform price is > 10% higher than historical lows.	Wait — price is currently marked up. Hold off for a sale.
BUY_COMPETITOR	Another model has a value score > 80, possibly higher on a particular model.	Consider higher competitor model instead for better hardware v
SKIP	Specs are weak (value score < 60) or price is singularly too high.	Skip this model due to poor specification-to-price ratio.

AI Interpretation & Summary Services

To present raw specs in an understandable format, the **DeviceSummaryService** and **InsightGenerator** parse technical benchmarks into written summaries. They generate heavy-use vs normal-use battery expectations, highlight camera low-light performance, and provide a single-sentence honest verdict, caching them in the ``phone_interpretations`` and ``phone_insights`` tables to avoid duplicate API calls to OpenAI.

5. Frontend Layout & Interactive Components

The user interface is built on Next.js 16 and is fully styled with tailwindcss to create a premium, futuristic dark mode theme. It leverages glassmorphic panel elements, custom gradients, and micro-interactions.

Search Hero with Autocomplete & Keyboard Navigation

The entry page (`frontend/app/page.tsx`) renders the `SearchHero` component. As users type, it calls `GET /api/discovery/search` with debouncing (300ms) to load results. It supports complete keyboard accessibility (ArrowDown, ArrowUp, Enter, Escape) to navigate the results list and highlights 'Best Match' badges if match scores exceed 90%.

Dynamic Variant Selector

Inside the product detail page, users can click variant chips for RAM, storage, or color. Choosing a different variant triggers dynamic price changes: it calls the backend prices endpoint with the `variant_id` query parameter, and recalculates the estimated launch price in real-time, adjusting discount gauges instantly.

Price Comparison & Inflation Indicator Panel

The `PriceComparisonSection` displays a listing table comparing all active merchants. If a listing has a high discount flag (inflated MRP), an 'Inflation Alert' icon is rendered. It calculates trust scores using seller reviews to help buyers judge merchant reliability.

Client-Side Fetch Fallback & Error Handling

To ensure robust client execution, page loading states are rendered using animated Pulse skeletons. Furthermore, server components include dynamic route safety: if a phone cannot be found in the database during metadata generation or initialization, the application captures the error, outputs diagnostics (such as the target slug), and presents a helpful debugging card indicating the root cause (e.g. missing API configuration or unseeded database).

6. Local Execution & Environment Setup

Follow these steps to run the complete stack locally for development and testing.

Step A — Backend Server Setup

```
# 1. Navigate to the backend directory
cd tech-decision/backend

# 2. Create the virtual environment
python -m venv .venv

# 3. Activate the environment
.venv\Scripts\activate

# 4. Install required dependencies
pip install -r requirements.txt

# 5. Set up environment variables (.env file)
# DATABASE_URL=sqlite:///./tech_decision.db
# FRONTEND_URL=http://localhost:3000
# OPENAI_API_KEY=your_key_here

# 6. Apply database schemas and migrations
alembic upgrade head

# 7. Seed sample phone data
python scripts/seed.py

# 8. Start uvicorn development server
python -m uvicorn main:app --host 127.0.0.1 --port 8000
```

Step B — Frontend Client Setup

```
# 1. Navigate to the frontend directory
cd tech-decision/frontend

# 2. Install Node dependencies
npm install

# 3. Configure local environment variables (.env.local)
# NEXT_PUBLIC_API_URL=http://localhost:8000
# NEXT_PUBLIC_API_BASE_URL=http://localhost:8000

# 4. Run next development server
npm run dev
```

Health Checks & API Verification

Verify the local services are running correctly by checking the health endpoint in your browser:
<http://localhost:8000/health>, which should return `{"status":"ok"}`. The swagger docs are interactive and accessible via <http://localhost:8000/docs>.